NotesIOE

notesioe.com

Compiled By

Aashish Lamsal

Theory Of Computation

Chapter 3 - CFG

Context-free Grammars

LH-12

Marks: 20-21

Introduction :-

⇒ we have learnt effective use of regular expressions for defining regular languages. But there are certain languages which cannot be defined by regular expressions eg. we cannot write a regular expression for checking well ~~defined~~ formed parenthesis etc. so in order to cover all these things and more complicated features leads us to the idea of a context-free ~~languages~~ Grammar for the languages.

⇒ while compiling the program, it is checked syntactically first and semantically latter. For checking the correctness of the program syntactically we need CfG.

* Defination :-

⇒ A context free grammar (CfG) is defined as 4-tuples (V_N, Σ, P, S) (S: starting non-terminal)

where, V_N = set of non-terminal symbol

Σ = set of terminal symbol

P = Production rule

S = starting non-terminal symbol.

Production rule is in the form,

one non-terminal → finite string of terminals and / or non-terminal.

i.e $A \to a$ where $A \in V_N$ and $a \in (V_N \cup \Sigma)^*$

* Application

⇒ The language generated by the context-free grammars are called context-free languages are applied in parser design.

⇒ It is also useful for describing block structure in programming languages.

⇒ CFGs are useful for describing arithmetic expressions, with arbitary nesting of balanced paranthesis.

* Derivations

⇒ The production rules are used to derive certain strings. The generation of language using specific rules is called derivation.

⇒ notations : $\Rightarrow$ and $\overset{*}{\Rightarrow}$

⇒ If $\alpha \to \beta$ be a production of P in CFG and a and b are strings in $(V_N \cup \Sigma)^*$ then $a\alpha b \Rightarrow a\beta b$.

Example 1 Construct a context-free grammar G generating all integers (with sign).

Soln Let $G = (V_N, \Sigma, P, S)$ where $V_N = \{S, \langle sign\rangle, \langle digit\rangle, \langle integer\rangle\}$

$\Sigma = \{0, 1, 2, 3 \ldots 9, +, -\}$

P consists of $S \to \langle sign\rangle \langle integer\rangle$

$\langle sign\rangle \to + | -$

$\langle integer\rangle \to \langle digit\rangle \langle integer\rangle | \langle digit\rangle$

$\langle digit\rangle \to 0|1|2|3|4|5|6|7|8|9.$

L(G) is the set of all integers.

L(G) is the language of G. It is the set of terminal strings that have derivations from the start symbol.

four components in CFG:-

1. There is the finite set of symbols that form the strings of Language being defined. We call this alphabet the terminal or terminal symbol.

2. There is a finite set of variables, also called non-terminal symbol. Each variable represent the Language i.e set of strings.

3. One of the variable represent the language being defined. It is called start symbol. It is denoted by S.

4. There is a finite set of rules or productions that represent the recursive defination of a language. Each productions consist of

(a) A variable that is being partially defined by the production. This variable is often called the head of production.

(b) The production symbol →

(c) A strings of zero or more terminals and variables. This string is called the body of production; represents one way to form strings in the language of the variable of the head

Note:-

↳ The variable symbols are represented by capital letters.

↳ The terminal symbols are represented by lowercase letters.

↳ One of the variable is designed as a start variable, It usually occurs in the left hand side of the production rule.

example

$S \to \varepsilon$

$S \to 0S1$

This is the CFG defining the grammar of all the strings with equal no. of 0's followed by equal no. of 1's.

Here, Two rules define the production P,

$\varepsilon, 0, 1$ are the terminal defining Σ or T,

S is the non-terminal symbol defining V.

and S is the start symbol from where production starts.

Example following is a CFG for the language $L = \{\omega c \omega^R | \omega \in (a,b)^*\}$

Solution Let G be the CFG for the language $L = \{\omega c \omega^R | \omega \in (a,b)^*\}$

$$G = (V_N, \Sigma, P, S)$$

Here, $V_N = \{S\}$

$\Sigma = \{a, b, c\}$

and P is given by

$S \to aSa$

$S \to bSb$

$S \to c$

let us check that abbcbba can be derived from given CFG

$S \Rightarrow aSa$ (use the $S \to aSa$)

$\Rightarrow abSba$ (use the $S \to bSb$)

$\Rightarrow abbSbba$ (use the $S \to bSb$)

$\Rightarrow abbcbba$ (use the $S \to c$)

Example2 Write a CFG, which generates string of balanced Parenthesis.

Solution This grammar will accept the balanced left and right Parenthesis. for example ( ) ( ) is acceptable. ((( ))) this is also acceptable.

let us design the CFG for this,

Let CFG be,

$G = (V_N, \Sigma, P, S)$ where

V_N = set of non-terminal = $\{S\}$

$\Sigma = \{(,)\}$

and set of production P is given by

$S \rightarrow SS$

$S \rightarrow (S)$

$S \rightarrow \varepsilon$

Now we see what this grammar generates

$S \Rightarrow SS$

$\Rightarrow (S)S$

$\Rightarrow (S)SS$

$\Rightarrow (S)(S)S$

$\Rightarrow (S)(S)(S)$

$\Rightarrow ()()()$

Thus above grammar will always generates balanced pair of parenthesis.

Example Write a CFG, which generates palindrome for binary numbers.

Solution Grammar will generate palindrome for binary numbers, that is 00, 010, 11, 101 ...

Let CFG be $G_1 = (V_N, \Sigma, P, S)$

V_N = set of non-terminals = $\{S\}$

Σ = set of terminals = $\{0, 1\}$

and P is the Production Rule defined as

$$S \to 0S0 \mid 1S1$$

$$S \to 0 \mid 1 \mid \varepsilon$$

Obviously this grammar generates palindrome for binary numbers it can be seen by the following derivation.

$$S \Rightarrow 0S0$$
$$\Rightarrow 01S10$$
$$\Rightarrow 010S010$$
$$\Rightarrow 01010010 \text{ which is palindrome.}$$

Example Write a CFG for a regular expressions.

$$r = 0^*1(0+1)^*$$

Solution Let us analyse regular expression

$$r = 0^*1(0+1)^*$$

Clearly regular expression is the set of strings which starts with any number of 0's followed by one and end with any combination of 0's and 1's.

Let CFG be $G = (V_N, \Sigma, P, S)$

where,

V_N = set of non-terminal symbol

$= \{S, A, B\}$

$\Sigma = \{0, 1\}$

and productions P are defined as

$S \rightarrow A1B$

$A \rightarrow 0A \mid \varepsilon$

$B \rightarrow 0B \mid 1B \mid \varepsilon$

Let us see the derivation of the string 00101

$S \Rightarrow A1B$

$\Rightarrow 0A10B$

$\Rightarrow 00A101B$

$\Rightarrow 00101$

Clearly, G is the CFG for regular expression r.

Example Write a CFG which generates strings having equal number of a's and b's.

Solution Let CFG be $G = (V_N, \Sigma, P, S)$

$V_N = \{S\}$

$\Sigma = \{a, b\}$

where P is defined as,

$S \rightarrow aSbS \mid bSaS \mid \varepsilon$

Let us derive a string $\omega = bbabaabbaa$

$S \Rightarrow bSaS$

$\Rightarrow bbSaSaS$

$\Rightarrow bbaSbSaSaS$

$\Rightarrow bbaSbaSbSaSaS$

$\Rightarrow bbaSbaaSbSbSaSaS$

$\Rightarrow bbabaaSbSbSaSaS$

$\Rightarrow bbabaaabaSbSaSaS$

$bbabaabbSaSaS$

$\Rightarrow$ ~~$bbabaababSaSaS$~~

$\Rightarrow bbabaabbaSaS$

$\Rightarrow bbabaabbaaS$

$\Rightarrow bbabaabbaa$

Example Design a CFG for the language $L = \{a^n b^m : n \neq m\}$

solution If $n \neq m$ then there are only two cases are possible

Case 1. $n > m$

let us say language L on condition $n > m$ is

L_1 and $L_2 = \{a^n b^m : n > m\}$

Let G_L be the CFG for the language L_1,

$G_L = (V_n^L, \Sigma^L, P^L, S^L)$

$V_N^L = \{S_1, A, S\}$

$\Sigma^L = \{a, b\}$

Then production P_1 are defined as,

$$S^1 \rightarrow AS_1$$

$$S_1 \rightarrow aS_1b \mid \epsilon$$

$$A \rightarrow aA \mid a$$

Case 2 $\quad n<m$

Let us say L on condition $n<m$ is L_2

$$L_2 = \{0^n 1^m : n<m\}$$

and Let CFG for G_2 be $G_{L2} = (V_N^2, \Sigma^2, P^2, S)$

$$V_N^2 = \{S^2, S_2, B\}$$

$$\Sigma^2 = \{a, b\}$$

P^2 are defined as,

$$S^2 \rightarrow S_2B$$

$$S_2 \rightarrow aS_2b \mid \epsilon$$

$$B \rightarrow bB \mid b$$

By combining G_L and G_2 we can write the CFG for L as

$$S \rightarrow S^1 \mid S^2$$

* Derivation

⇒ CFG generates string according to the following process called derivation.

(a) We start by writing down the start symbol of the CFGs.

(b) At each step of the derivation, we may replace any non-terminal symbol of the string generated so far by the right hand side of any production that has the symbol on the left.

(c) The process ends when it is impossible to apply a step of type described in (b).

⇒ If the string at the end of this process consists entirely of terminals, it is a string in the language generated by the grammar called yield.

⇒ There are two approaches of derivation

1. Body to head approach (Bottom up)
2. Head to Body (Top down)

1. Body to head :-

⇒ Here, we take strings known to be in the language of each of the variables of the body, concatenate them in the proper order with any terminals appearing in the body, the resulting string is the language of the variable in the head.

Consider grammar,

$S \to S+S$

$S \to S|S$

$S \to (S)$

$S \to S-S$

$S \to S*S$

$S \to a$

Here given a+(a*a)|a-a

now, applying body to head approach,

S.N	String inferred	Variable	Production	string used.
1.	a	S	S→a	
2.	a*a	S	S→S*S	string ①
3.	(a*a)	S	S→(S)	string ②
4.	(a*a)|a	S	S→S|S	string ③ and ①
5.	(a*a)|a-a	S	S→S-S	string ④ and ①
6.	a+(a*a)|a-a	S	S→S+S	string ⑤ and ①

Thus in this approach we start with any terminal appearing in the body and use the available rules from body to head.

2. Head to Body :-

Here, we used production from head to body. we expand the start symbol using a production, whose head is the start symbol. Here we expand the resulting string until all strings of terminal are obtained. Here we have two approaches

(a) Left most Derivation :-

Here, leftmost symbol (variable) is replaced first

(b) Right most Derivation :-

Here, rightmost symbol is replaced first.

Example Consider the grammar G1 with production

$$S \rightarrow aSS \mid b$$

Now, we have

$$\begin{aligned} S &\Rightarrow aSS \\ &\Rightarrow aaSSS \\ &\Rightarrow aabSS \\ &\Rightarrow aabaSSS \\ &\Rightarrow aababSS \\ &\Rightarrow aababbS \\ &\Rightarrow aababbb \end{aligned}$$

The sequence followed is left-most derivation

Now right-most derivation:

$$\begin{aligned} S &\Rightarrow aSS \\ &\Rightarrow aSb \\ &\Rightarrow aaSSb \\ &\Rightarrow aaSaSSb \\ &\Rightarrow aaSaSbb \\ &\Rightarrow aaSabbb \\ &\Rightarrow aababbb \end{aligned}$$

Parse Tree or Derivation Tree

⇒ The strings generated by a context free grammar (V_N, Σ, P, S) can be represented by a hierarchical structure called tree. Such trees representing derivations are called derivation trees or parse tree.

⇒ characteristics of a Parse Tree

A parse tree for a context-free grammar G has the following characteristics

1. Every vertex of a parse tree has a label which is a variable or terminal or ε.
2. The root of parse tree has label S (start symbol).
3. The label of an internal vertex is a variable.
4. If a vertex A has k children with labels $A_1, A_2 \ldots A_k$ then $A \to A_1, A_2, \ldots A_k$ will be a production in context free Grammar G.
5. A vertex 'n' is a leaf if its label is $a \in \Sigma$ or ε.
6. 'n' is the only son of its father if its label is ε.

Example Derive the string "aabbaa" and construct a derivation tree for "aabbaa" for a grammar G with productions $S \to aAS|a, A \to SbA|SS|ba$

Solution

$S \to aAS|a$

$A \to SbA|SS|ba$

String "aabbaa" is derived from S as,

$S \Rightarrow aAS$

$\Rightarrow aSbAS$

$\Rightarrow aaSbAS$

$\Rightarrow aabbaS$

$\Rightarrow aabbaa$

The derivation tree is shown as.

Example Let $G = (V, \Sigma, P, S)$ be a grammar. where.

$V = \{E\}$

$\Sigma = \{+, *, a, b\}$

$S = E$

P consist of

$E \Rightarrow E+E \mid E*E \mid a \mid b$

Soln

LMD

$S \Rightarrow E$

$\Rightarrow E+E$

$\Rightarrow a+E$

$\Rightarrow a+E*E$

$\Rightarrow a+b*E$

$\Rightarrow a+b*E*E$

$\Rightarrow a+b*a*E$

$\Rightarrow a+b*a*b$

RMD

$S \Rightarrow E$

$\Rightarrow E*E$

$\Rightarrow E*b$

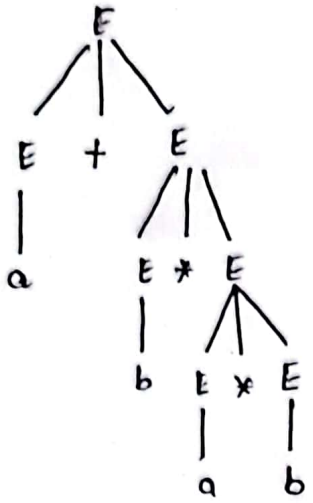
$\Rightarrow b*E*b$

$\Rightarrow E*a*b$

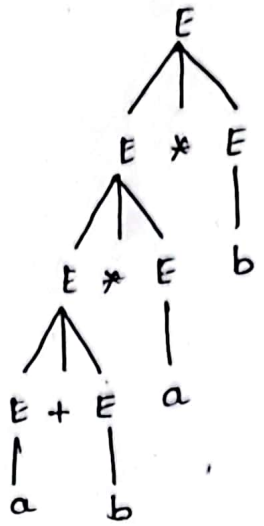
$\Rightarrow E+E*a*b$

$\Rightarrow E+b*a*b$

$\Rightarrow a+b*a*b$



LMD parse tree



RMD parse tree

Example Consider the grammar G.

$S \rightarrow A1B$

$A \rightarrow 0A|\varepsilon$

$B \rightarrow 0B|1B|\varepsilon$

1. Construct the parse tree for 00101.

LMD

$S \Rightarrow \underline{A}1B$

$\Rightarrow 0\underline{A}1B$

$\Rightarrow 00\underline{A}1B$

$\Rightarrow 00\varepsilon 1B$

$\Rightarrow 00\varepsilon 10B$

$\Rightarrow 00\varepsilon 101\varepsilon$

$\Rightarrow 00101$

RMD

$S \Rightarrow A1\underline{B}$

$\Rightarrow A10\underline{B}$

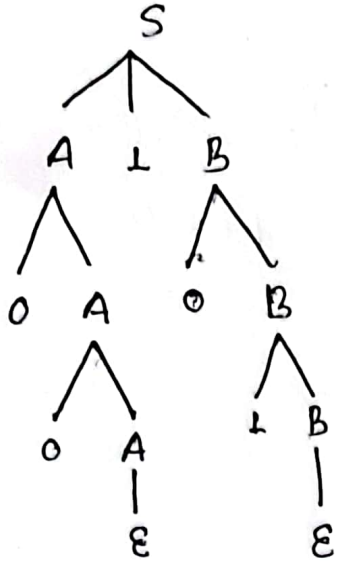
$\Rightarrow A101\underline{B}$

$\Rightarrow \underline{A}101\varepsilon$

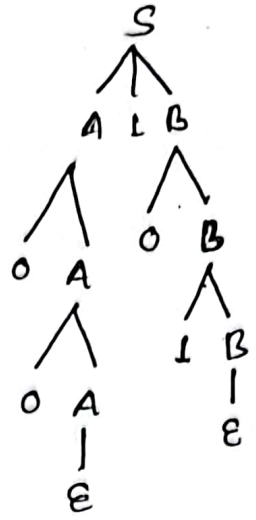
$\Rightarrow 0A101$

$\Rightarrow 00A101$

$\Rightarrow 00101$



LMD parse tree



RMD parse tree

* Ambiguity in Grammar:-

⇒ A grammar $G = (V, \Sigma, P, S)$ is said to be ambiguous if there is a string $\omega \in L(G)$ for ~~each~~ which we can derive two or more distinct derivation tree at S and yielding ω. In other words, a grammar is ambiguous if there exist more than one leftmost derivation or more than one rightmost derivation for the same string.

Example

$S \to AB \mid aaB$

$A \to a \mid Aa$

$B \to b$

for any string aab;
we have two leftmost derivation as.

$S \to \underline{A}B$

$\to AaB$

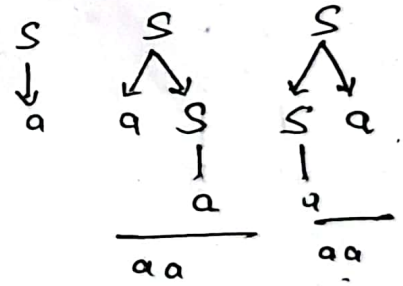
$\to aaB$

$\to aab$

Also, $S \to aaB$

$\to aab$

$S \to aS \mid Sa \mid a$



Here, we see there are two ways to generate string aa. Thus ambiguous.

for example

$G = (\{S\}, \{a, b, +, *\}, P, S)$ where P consist

$S \to S+S \mid S*S \mid a \mid b$, there are two derivation trees.

For a+a*b

Soln ~~two have two left most derivation tree~~

$S \to S+S$

$\to a+S$

$\to a+S*S$

$\to a+a*S$

$\to a+a*b$

Also,

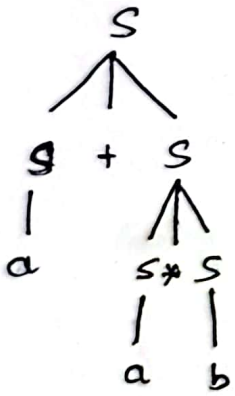
$S \to S*S$

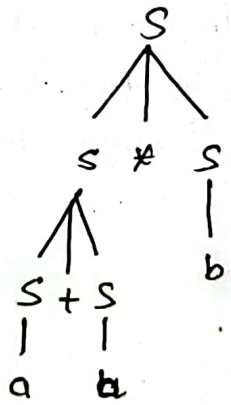
$\to S*b$

$\to S+S*b$

$\to S+a*b$

$\to a+a*b$





Thus, the given grammar is ambiguous.

* Inherent Ambiguity :-

⇒ If every grammar that generates L is ambiguous, then the language is said to be "inherently ambiguous".

Let us consider an example for it

Show that $L = \{a^n b^n c^m\} \cup \{a^n b^n c^m\}$ with n and m are non-negative, is an inherently ambiguous context-free language

Soln Let us say $L = L_1 \cup L_2$

where, $L_1 = \{a^n b^n c^m\}$

and $L_2 = \{a^n b^n c^m\}$

Let us write CFG for L_1, Let it is G_{L_1} as follows:

$$S_1 \to S_L c \mid A$$

$$A \to aAb \mid \varepsilon$$

and G_{L_2} is as follows:

$$S_2 \to aS_2 \mid B$$

$$B \to bBc \mid \varepsilon$$

Then L is generated by the combination of these two Grammars with additional production

$$S \to S_1 \mid S_2$$

This grammar L is ambiguous since the string $a^n b^n c^n$ has two different derivations one starting with $S \to S_1$ and another with $S \to S_2$

Here L_1 and L_2 have conflicting requirements, the first putting a restriction on the number of a's and b's while second does the same for b's and c's. Few tries will quickly convince us of the impossibility of combining these requirements in a single set of rules. So this is inherently unambiguous grammar.

* Removal of Ambiguity

⇒ In programming Languages, where there should be only one interpretation of each statement, ambiguity must be removed when possible.

⇒ often we can achieve this by rewriting the grammar in an equivalent unambiguous form.

Example. Grammar

$E \to I$

$E \to E+E$

$E \to E*E$

$E \to (E)$

$I \to a|b|c$

is ambiguous and the ambiguity is due to the precedence of operator. If we correct the precedence then ambiguity may be removed.

Here, two causes of ambiguity in the grammar.

1. The precedence of operators is not respected.

2. The sequence of identical operators can group either from left or from the right. We would see two different parse trees for $E+E+E$ Since addition is associative, it doesnot mater whether we group from the left or the right, but to remove ambiguity we must take one.

The unambiguous grammar is

$E \Rightarrow T$

$T \Rightarrow F$

$F \Rightarrow I$

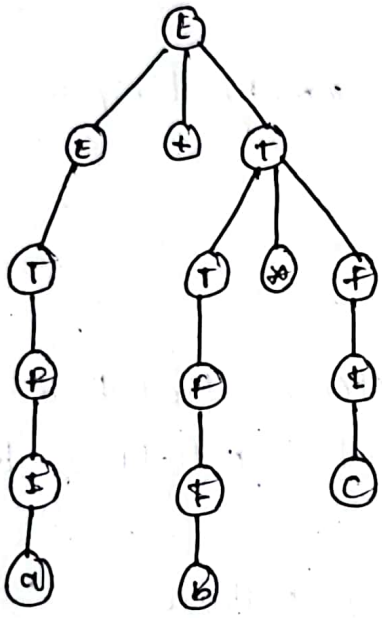
$E \Rightarrow E+T$

$T \Rightarrow T*F$

$F \Rightarrow (E)$

$I \Rightarrow a|b|c$

The sole parse tree for a+b*c is



* Simplification of CFG:-

⇒ There are mainly 3 simplifications of CFGs

1. Elimination of epsilon production
2. Elimination of unit production
3. Elimination of useless symbol

1. Elimination of epsilon production:-

⇒ A production of the form

non-terminal → empty string is called null Production.

eg. $S \to \varepsilon$, $A \to \varepsilon$ etc.

⇒ To eliminate ε-production from CFG, we first find out nullable non-terminal of CFG. A non-terminal is nullable if it derives an epsilon in zero or more steps.

i.e $A \overset{*}{\to} \varepsilon$ then A is nullable

⇒ Then we search for occurrence of the non-terminals that generates empty string i.e in all other production rule.

⇒ If such occurrence of the non-terminal exists then we replace that non terminal with the empty string and add the resulting production to the grammar.

⇒ After adding the new production to the grammar, we discard the null production from the grammar. The process is continued until all the null productions are eliminated.

Example :- Consider the following grammar G.

$S \rightarrow ABAC$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow bB \mid \varepsilon$

$C \rightarrow c$

Remove the epsilon from the grammar.

Solution Here, null production are,

$A \rightarrow \varepsilon$ and $B \rightarrow \varepsilon$

for $A \rightarrow \varepsilon$,

Since the occurence of A oxists in the production

$S \rightarrow ABAC$

We replace 'A' by ε in all ways.

$S \rightarrow ABAC$

$\rightarrow \varepsilon BAC$

$\rightarrow BAC$

$S \rightarrow ABAC$

$\rightarrow ABeC$

$\rightarrow ABC$

$S \rightarrow ABAC$

$\rightarrow \varepsilon B \varepsilon A$

$\rightarrow B$

and $A \rightarrow aA$

$\rightarrow a\varepsilon$

$\rightarrow a$

Adding new productions to grammar.

$S \rightarrow ABAC \mid ABC \mid BAC \mid BC$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid \varepsilon$

$C \rightarrow c$

for $B \to \varepsilon$,

$S \to ABAC$ $S \to ABC$ $S \to BAC$ $S \to BC$

$\to AεAC$ $\to AεC$ $\to εAC$ $\to εC$

$\to AAC$ $\to AC$ $\to AC$ $\to C$

and $B \to bB$

$\to bε$

$\to b$

∴ final grammar is,

$S \to ABAC|ABC|BAC|BC|AAC|AC|C$

$A \to aA|a$

$B \to bB|b$

$C \to C$

Example Consider the grammar:

$S \to ABC$

$A \to BB|\varepsilon$

$B \to εC|a$

$C \to AA|b$

Solution Here $A \to \varepsilon$, A is nullable

$C \to AA \overset{*}{\Rightarrow} \varepsilon$, C is nullable

$B \to CC \Rightarrow \varepsilon$, B is nullable.

$S \to ABC \overset{*}{\Rightarrow} \varepsilon$, S is nullable

for $A \to \varepsilon$,

$S \to ABC$

$\to BC$

now adding these to grammar,

$S \rightarrow ABc|Bc$

$A \rightarrow BB$

$B \rightarrow cc|a$

$C \rightarrow AA|b$

for $C \rightarrow AA$,

$C \rightarrow AA$	$C \rightarrow AA$	$C \rightarrow AA$
$\rightarrow \varepsilon A$	$\rightarrow A\varepsilon$	$\rightarrow \varepsilon\varepsilon$
$\rightarrow A$	$\rightarrow A$	$\rightarrow \varepsilon$

adding these to grammar.

$S \rightarrow ABc|Bc$

$A \rightarrow BB$

$B \rightarrow cc|a$

$C \rightarrow AA|A|\varepsilon|b$

$\because C \rightarrow \varepsilon$,

$S \rightarrow ABC$	$S \rightarrow BC$	$B \rightarrow CC$	$B \rightarrow C$
$\rightarrow AB\varepsilon$	$\rightarrow B\varepsilon$	$\rightarrow C\varepsilon$	$\rightarrow \varepsilon\varepsilon$
$\rightarrow AB$	$\rightarrow B$	$\rightarrow C$	$\rightarrow \varepsilon$

now, adding these to grammar,

$S \rightarrow ABc|Bc|AB|B$

$A \rightarrow BB$

$B \rightarrow cc|\varepsilon|a$

$C \rightarrow AA|a|b$

Again,

for $B \to \varepsilon$,

$S \to ABC \to A\varepsilon C \to AC$

$S \to BC \to \varepsilon C \to C$

$S \to AB \to A\varepsilon \to A$

$S \to B \to \varepsilon$

$A \to BB \to B\varepsilon \to B$

$A \to BB \to \varepsilon B \to B$

$A \to BB \to \varepsilon\varepsilon \to \varepsilon$

Now the grammar becomes,

$S \to ABC|BC|AB|B|AC|C|A|\varepsilon$

$A \to BB|B|\varepsilon$

$B \to cc|c|a$

$C \to AA|A|b$

for $S \to \varepsilon$, no occurrence of non-terminal that generate empty string i.e other production rules.

∴ final CFG :-

$S \to ABC|BC|AB|B|AC|C|A$

$A \to BB|B$

$B \to cc|c|a$

$C \to AA|A|b$

2. Eliminating unit production

⇒ A production of the form, non-terminal → non-terminal is called unit production.

eg:- $S \to A$, $A \to B$ etc.

⇒ To eliminate such production rules from the given CFG, we find all the unit production of the form $\alpha \to \beta$, where $\alpha, \beta \in V$. Then we search production of the form $\beta \to a$, where $\beta \in V$ and $a \in T$

⇒ If such production rule exists then we replace the right hand side non-terminal of the unit production with respective terminal.

i.e we add the production $\alpha \to a$, where $\alpha / A \in V$ and $a \in T$ to the grammar and discard $\alpha \to \beta$ from the grammar.

⇒ We continue the process until all the unit productions are eliminated from CFG.

Example Consider the CFG as,

$S \to AB$

$A \to a$

$B \to C/b$

$C \to D$

$D \to E$

$E \to a$

Remove the unit production.

Solution Here, the unit production of the given grammar are:-

$B \to C$

$C \to D$

$D \to E$

⇒ Now we have to search for the production rule such that either of C, D or E gives a terminal symbol. Among these unit productior

the right hand side non-terminal of unit production $D \to E$ gives a terminal symbol 'a'.

i.e $E \to a$ exists in the given grammar.

⇒ Now we replace E by a in the unit production $D \to E$ such that a new production $D \to a$ is added to the grammar and $D \to E$ is discarded from the given grammar.

$S \to AB$

$A \to a$

$B \to \in | b$

$C \to D$

$D \to a$

$E \to a$

Also, $C \to D$

$S \to AB$

$A \to a$,

$B \to C | b$

$C \to a$

$D \to a$

$E \to a$

∵ $D \to a$

∴ $C \to a$

∵ $C \to a$

Also, $B \to C$

∴ $B \to a$

⇒ Final grammar after removing unit production is:

$S \to AB$

$A \to a$

$B \to a | b$

Q. Consider the CFG G given below. Find a CFG 'G' with no ε-production and no unit production.

$S \to ABA$

$A \to aA \mid \varepsilon$

$B \to bB \mid \varepsilon$

Solution Here,

$A \to \varepsilon$

$B \to \varepsilon$

are null productions

⇒ For $A \to \varepsilon$

The occurrence of A exists in the production as

$S \to ABA$ $S \to ABA$ $S \to ABA$

$\to BA$ $\to AB$ $\to B$

Also, $A \to aA$

$\to a$

∴ After removing $A \to \varepsilon$ production, we add these new productions to grammar.

$S \to ABA \mid BA \mid AB \mid B$

$A \to aA \mid a$

$B \to bB \mid \varepsilon$

⇒ For $B \to \varepsilon$

$S \to ABA$ $S \to BA$ $S \to AB$ $S \to B$

$\to AA$ $\to \varepsilon A$ $\to A\varepsilon$ $\to \varepsilon$

Also,

$B \to bB$

$\to bE$

$\to b$

∴ After removing $B \to E$, we add these new productions to the grammar.

$S \to ABA \mid BA \mid AB \mid B \mid AA \mid A \mid E$

$A \to aA \mid a$

$B \to bB \mid b$

For $S \to E$, no occurrence of S. So simply remove $S \to E$.

∴ final production after removing null productions are!

$S \to ABA \mid BA \mid AB \mid AA \mid A \mid B$

$A \to aA \mid a$

$B \to bB \mid b$

Again, In the above CFG, unit productions are.

$S \to A$

$S \to B$

⇒ Now we have to search a production rule such that either of A, B gives a terminal symbol.

i.e $A \to a$

$B \to b$

Exists in the given grammar.

⇒ Now we replace 'B' by b and 'A' by 'a' in the unit production.

$S \to A$ and $S \to B$

such that $S \to a$ and $S \to b$ is added to the grammar and $S \to B$ and $S \to A$ is discarded from the grammar.

⇒ The final CFG after removal of unit production is

$S \to ABA \mid BA \mid AB \mid AA \mid a \mid b$

$A \to aA \mid a$

$B \to bB \mid b$

3. Eliminating useless Symbol:-

⇒ A symbol X is said to be useful if

(i) X is generating, if X is of the form

$$X \xrightarrow{*} \omega, \quad \omega \in T$$

i.e If X is generating, it gives a terminal symbol or symbols after using series of production rule.

(ii) X is reachable from the start symbol, if X is of the form

$$S \xrightarrow{*} \alpha X \beta$$

⇒ A symbol that doesnot satisfy these two conditions is called useless symbol.

⇒ To eliminate useless symbol from the grammar, we first find all the generating symbol in the given grammar i.e all the symbols that generates a terminal symbol. If any symbol doesnot terminate then discard this symbol from the grammar.

⇒ There may be some symbols that generates terminal symbol or string but they are not reachable from the start symbol. Such symbols have no roles in string generating. Therefore, we have to discard this as well.

⇒ To eliminate the non-reachable symbol, we draw a dependency graph using these symbols. Any symbol that remains unconnected in a graph is non-reachable symbol and we discard it from grammar.

⇒ Thus after eliminating the non-generating symbol's and unreachable symbols, we shall have only the useful symbol left.

Note:-

↳ whenever we have to simplify CFG, we first make sure the CFG is

- ε-production free
- unit-production free

↳ Then only we apply elimination of useless symbol.
↳ We must follow this order of processing for simplification of CFG.

Example Consider the grammar G;

$$S \rightarrow aB \mid bX$$

$$A \rightarrow Bad \mid bSX \mid a$$

$$B \rightarrow aSB \mid bBX$$

$$X \rightarrow SBd \mid aBX \mid ad$$

Eliminate the useless symbol.

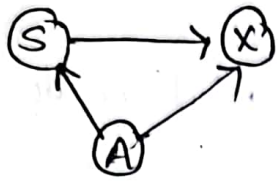
Solution

Here, S, A, X are only generating symbols. Therefore, remove B from the grammar. Since B is not generating.

⇒ The new grammar after removing non-generating symbol is

$$S \to bX$$
$$A \to bSX \mid a$$
$$X \to ad$$

⇒ Now, to remove the non-reachable symbol.



⇒ It is seen clearly that from start symbol S we cannot reach A. So A is unreachable. The simplified grammar is

$$S \to bX$$
$$X \to ad$$

Example Find a CFG equivalent to

$$S \to AB \mid CA$$
$$A \to a$$
$$B \to BC \mid AB$$
$$C \to aB \mid b$$

with no useless symbol.

Solution

Here A and C are generating symbols, since they can directly generate terminal symbol.

Also, 'S' can produce terminals. Since $S \rightarrow CA$. where both C and A can produce terminals.

⇒ But B doesnot produce any terminal symbol. Therefore, B is non-generating. Thus we eliminate B from the CFG.

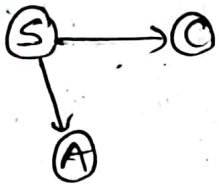
The remaining CFG becomes,

$S \rightarrow CA$

$A \rightarrow a$

$C \rightarrow b$

⇒ Now for removing un-reachable symbol, we draw dependency graph as.



⇒ In the above CFG, there is no any symbol which is unreachable

∴ final CFG is,

$S \rightarrow CA$

$A \rightarrow a$

$C \rightarrow b$

* Normal forms for CFG :-

⇒ In a CFG, the R.H.S of a production can be any string of variable and terminals.

⇒ When the productions in G satisfy certain restriction then G is said to be in 'normal form'.

⇒ Among several normal form two important are

(i) Chomsky Normal form (CNF)

(ii) GREIBACH Normal form (GNF)

1. Chomsky Normal Form (CNF)

⇒ A CFG G is said to be in CNF if every production is of the form either $A \to a$ or $A \to BC$ and $S \to \varepsilon$ if $\varepsilon \in L(G)$.

* Reduction to CNF :-

⇒ Let us consider, a CFG having productions $S \to AB/aC$, $A \to b$, $B \to aA$ and $C \to ABS$.

⇒ In these, except $S \to aC$, $B \to aA$, $C \to ABC$, all the other productions are in the form required to be in CNF.

⇒ For the production of the form $S \to aC$, we introduce new variable C_a such that $S \to aC$ can be replaced by $S \to C_aC$, $C_a \to a$ which are in the form required for CNF.

⇒ Similarly, $B \to aA$ can be replaced by $B \to C_aA$ and there is no need to write $C_a \to a$ again.

⇒ For the production of the form $C \to ABS$, we introduce new variable D such that $D \to BS$ so $C \to AD$ which are in CNF.

Let $G_1 = (V_N, \Sigma, P, S)$ be a CFG. A context-free grammar is converted in chomsky normal form by using following steps

step 1: Eliminate of null and unit productions.

step 2: Eliminate of terminal from right-hand side

step 3: Restricting the number of variables on the right-hand side.

Example :- find the grammar in Chomsky Normal form equivalent to $S \to aAbB$, $A \to aA|a$, $B \to bB|b$.

Solution :-

Step 1: Eliminate all null and unit productions.

There is no such productions so we proceed to step 2.

Step 2: Elimination of terminals from R.H.S

(i) All the productions of the form Non-terminal → terminal or Non-terminal → Non-terminal Non-terminal are included.

(ii) If there is a productions of the form $X \to A_1 A_2 \ldots A_m$ with some terminals on R.H.S. If A_i is a terminal, say a_i, add a new variable C_{a_i} to V_N' and $C_{a_i} \to \alpha_p$ to P_1. In production $X \to A_1 A_2 \ldots A_n$ every terminals on R.H.S is replaced by corresponding new variables on the R.H.S are retained and the resulting production is added to P_1.

Here, $A \to a$, $B \to b$ are added to P_1.

$S \to aAbB$, $A \to aA$, $B \to bB$ yields $S \to C_a A C_b B$

$C_a \to a$, $C_b \to b$, $A \to C_a A$, $B \to C_b B$ and

$V_N = \{S, A, B, C_a, C_b\}$

Step 3 Restricting the number of variables on R.H.S.

Consider $A \to A_1 A_2 \ldots A_m$ where $m \geq 3$. We introduce new productions $A \to A_1 C_1$ $C_1 \to A_2 C_2$ $C_2 \to A_3 C_3 \ldots C_{m-2} \to A_{m-1} C_m$ and new variables $C_1, C_2, \ldots C_{m-2}$. These are added in P' and V_N' respectively.

Here P_1 consists of $S \to C_a A C_b B$. $A \to C_a A$, $B \to C_b B$, $C_a \to a$, $C_b \to b$, $A \to a$, $B \to b$.

$S \to C_a A C_b B$ is replaced by $S \to C_a C_1$, $C_1 \to A C_2$, $C_2 \to C_b B$ and remaining productions are added to production P_2. So

$$G_{12} = (\{S, A, B, C_a, C_b, C_1, C_2\}, \{a, b\}, P_2, S)$$

where P_2 consist of

$S \to C_a C_1$, $C_1 \to A C_2$, $C_2 \to C_b B$, $A \to C_a A$, $B \to C_b B$, $C_a \to a$, $C_b \to b$, $A \to a$, $B \to b$.

which is in CNF and equivalent to given grammar.

Method 2

Solution Given CFG is already simplified.

Cycle 1 :

(i) Production $A \to a$ and $B \to b$ are already in CNF.

(ii) Consider the production $S \to aAbB$, we can re-write as

$S \to V_1 A V_2 B$ where $V_1 \to a$, $V_2 \to b$ and V_1 and V_2 are new non-terminals.

(iii) Consider the production $A \to aA$, we can re-write as,

$A \to V_1 A$

(iv) Consider the production $B \to bB$ we can re-write as,

$B \to V_2 B$

Thus from (i), (ii), (iii) and (iv)

$S \to V_1 A V_2 B$

$A \to V_1 A \mid a$

$B \to V_2 B \mid b$

$V_1 \to a$

$V_2 \to b$

Cycle 2

(v) Production $V_1 \to a$, $V_2 \to b$, $A \to V_1 A$ and $B \to V_2 B$ are already in CNF.

(i) Consider the production $S \to V_1 A V_2 B$, we can rewrite as

$S \to V_3 V_4$ where $V_3 \to V_1 A$ and $V_4 \to V_2 B$

Thus the final CFG in CNF is

$S \to V_3 V_4$

$V_3 \to V_1 A$

$V_4 \to V_2 B$

$V_1 \to a$

$V_2 \to b$

$A \to V_1 A \mid a$

$B \to V_2 B \mid b$

Example 2 Consider an example

$S \to AAC$

$A \to aAb \mid \epsilon$

$C \to ac \mid a$,

Convert the above grammar into CNF.

Solution

(a) Remove ε-productions;

$A \to \epsilon$, hence A is nullable

Now,

$S \to AAC$, $S \to AAC$ $S \to AAC$

$\to \epsilon AC$ $\to A\epsilon C$ $\to \epsilon\epsilon C$

$\to AC$ $\to AC$ $\to C$

Also, $A \to aAb$

$\to a\epsilon b$

$\to ab$

∴ After eliminating ε-productions we have

$S \to AAC \mid AC \mid C$

$A \to aAb \mid ab$

$C \to ac \mid a$

(b) Remove unit production

Here, unit production is $S \to C$

Hence, unit pair is (S, C)

$S \to C \to ac$

$S \to C \to a$

After removing unit production $S \to C$, we have

$S \to AAC | AC | acla$

$A \to aAb | ab$

$C \to ac | a$

© Removing useless symbols.

Here. S, A, C are non-terminals and all are generating.
Since they all provide terminal productions. Also all these non-terminal symbols are reachable. Hence, we donot have any useless symbols.
Now we can convert the grammar into CNF.

Cycle 1:

(i) Productions $S \to Ac | a$ and $C \to a$ are already in CNF.

(ii) Consider the productions $S \to aC$, we can rewrite as

$S \to V_1 C$, where $V_1 \to a$

Also, $A \to aAb$, we can rewrite as

$A \to V_1 A V_2$, where $V_2 \to b$

also, $A \to ab$ can written as

$A \to V_1 V_2$

also, $C \to ac$ can be written as

$C \to V_1 C$

Thus, $S \to AAC | AC | V_1 C | a$

$V_1 \to a$

$A \to V_1 A V_2 | V_1 V_2$

$C \to V_1 C | a$

Cycle 2

(iii) for $S \to AAC$, we can rewrite as

$S \to V_3C$, $V_3 \to AA$

for $A \to V_1AV_2$, we can rewrite as

$A \to V_4V_2$, $V_4 \to V_1A$

Thus the final grammar in CNF will be as

$S \to V_3C \mid AC \mid V_1C \mid a$

$V_3 \to AA$

$V_1 \to a$

$A \to V_4V_2 \mid V_1V_2$

$V_2 \to b$

$C \to V_1C \mid a$

$V_4 \to V_1A$

2. Greibach Normal Form (GNF) :-

⇒ A Grammar $G = (V, \Sigma, P, S)$ is said to be in GNF if its production rules are of the form

$$\alpha \to a\beta^*$$

where $a \in \Sigma$

$\beta \in V$

i.e non-terminal → exactly one terminal OR

non-terminal → one terminal followed by any no. of non-terminals

Pumping Lemma for Context-free Language :-

⇒ Pumping Lemma is used to prove that a language is not regular but we can't prove that the language is regular.

⇒ Similarly, in the case of context-free Language we cannot prove that a context-free language and the pumping lemma given are only for finite languages.

⇒ If L is any context-free language, then we can find a natural number 'n' such that:

(i) every $w \in L$ where $|w| \geq n$ can be written as $uvwxy$ for some strings u, v, w, x and y.

(ii) $|vx| \geq 1$

(iii) $|vwx| \leq n$

(iv) $uv^i wx^i y \in L \quad \forall\ i \geq 0$

Example :- Show that $L = \{a^n b^n c^n \mid n \geq 1\}$ is not a CFL.

Soln: Let the given Language 'L' is a context-free Language.

Let $n = 3$

$L = \{abc, aabbcc, aaabbbccc \ldots\}$

$w = aaabbbccc$

checking (1) case :- $w = aaabbbccc$

$|w| \geq n$

$|aaabbbccc| \geq 3$

$9 \geq 3$, which is true.

checking 2 case :

$z = aaabbbccc$

$u\ v\ w\ x\ y$

$u = aa,\ v = a,\ w = b,\ x = b,\ y = ccc$

$|vwx| \le n \Rightarrow |a.b.b| \le 3$

$3 \le 3$

checking 3 case :-

i.e $|vx| \ge 1$

$|ab| \ge 1$

$2 \ge 1$, true

For $i = 0$, $uv^i wx^i y \in L$ is in L.

$aa.a^0.b.b^0.bccc$

$\Rightarrow aabbccc \notin L$, which is false

Thus, the given Languge is not CFL.

Closure Properties of CFL:-

⇒ Whenever we apply operations like union, intersection, concatenation, complement, star closure etc on given CFL and the result of these operations are again a CFL, then we say that the family of CFL is closed under the operations done.

⇒ The CFL's are closed under some operation means 'after performing that particular operation on those CFLs the resultant language is CFL. These properties are:

1. The context free languages are closed under union.
2. The context free languages are closed under concatenation.
3. The context free Languages are closed under kleen closure.
4. The context free Languages are not closed under intersection.
5. The context free Languages are not closed under complement.

Theorem: If L_1 and L_2 are context-free Languages then $L = L_1 \cup L_2$ is also context-free. That is, the CFLs are closed under union.

Proof:- We will consider two Languages L_1 and L_2 which are context-free languages. We can give these Languages using context-free Grammars G_1 and G_2 such that $G_1 \in L_1$ and $G_2 \in L_2$. The G_1 can be given as

$G_1 = \{V_1, \Sigma, P_1, S_1\}$ where P_1 can be given as,

$$P_1 = \{ S_1 \to A_1 S_1 A_1 \mid B_1 S B_1 \mid \varepsilon$$

$$A_1 \to a$$

$$B_1 \to b$$

Here $V_1 = \{S_1, A_1, B_1\}$ and S_1 is a start symbol.

Similarly, we can write $G_2 = \{V_2, \Sigma, P_2, S_2\}$

$V_2 = \{S_2, A_2, B_2\}$ and S_2 is a start symbol.

P_2 can be given as:

$P_2 = \{ S_2 \to aA_2A_2 \mid bB_2B_2$

$A_2 \to b$

$B_2 \to a$

$\}$

Now, $L_1 \cup L_2$ gives $G \in L$ This G can be written as.

$G = \{V, \Sigma, P, S\}$

$V = \{S_1, A_1, B_1, S_2, A_2, B_2\}$

$P = \{P_1 \cup P_2\}$

S is a start symbol.

$P = \{$

$S \to S_1 \mid S_2$

$S_1 \to A_1S_1A_1 \mid B_1SB_1 \mid \varepsilon$

$A_1 \to a$

$B_1 \to b$

$S_2 \to aA_2A_2 \mid bB_2B_2$

$A_2 \to b$

$B_2 \to a$

$\}$

Thus grammar G is context-free Grammar which produces Languages L which is context free Language.

Theorem 2

If L_1 and L_2 are two context free Language then $L = L_1L_2$ is also context free. That means context free language are closed under concatenation.

Proof:-

Let L_1 is a context free language which can be represented by a context-free grammar G_1, such that $G_1 \in L_1$ and

$$G_1 = \{ V_1, \Sigma, P_1, S_1 \}$$

$$V_1 = \{S_1, A_1, B_1\}$$

$$\Sigma = \{ a, b \}$$

S_1 is a start symbol and P_1 is a set of production rules.

$$P_1 = \{ S_1 \to A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid e$$

$$A_1 \to a$$

$$B_1 \to b$$

$$\}$$

Similarly, L_2 is a context-free language which can be represented by a context-free grammar G_2, such that $G_2 \in L_2$ and

$$G_2 = \{ V_2, \Sigma, P_2, S_2 \}$$

$$V_2 = \{ S_2, A_2, B_2 \}$$

S_2 is a start symbol and P_2 is a set of production rules.

$$P_2 = \{$$

$$S_2 \to a A_2 A_2 \mid b B_2 B_2$$

$$A_2 \to b$$

$$B_2 \to a$$

$$\}$$

Now $L = L_1 L_2$ can be obtained by G_1 such that $G_1 = G_{L1} \cdot G_{L2}$.

Therefore,

$G_1 = \{V, \Sigma, P, S\}$

$V = \{S, S_1, A_1, B_1, S_2, A_2, B_2\}$

The production rule P can be given as.

$P = \{$

$S \to S_1 | S_2$

$S_1 \to A_1 S_1 A_1 | B_1 S_1 B_1 | \varepsilon$

$A_1 \to a$

$B_1 \to b$

$S_2 \to a A_2 A_2 | b B_2 B_2$

$A_2 \to b$

$B_2 \to a$

$\}$

As grammar G_1 is a context free grammar the language L produced by G_1 is also context free language. Hence, context free language are closed under concatenation.

Theorem 3 If L_1 is the context-free language then L_1^* is also context free. That means CFL is closed under kleene closure.

Proof:-

Let L_1 be a context free language represented by $G_{L1} \to \varepsilon L_1$

The CFG G_{L1} can be given as,

$G_{L1} = \{V_1, \Sigma, P, S_1\}$

where S_1 is a start symbol and

$P_1 = \{ S_1 \to A_1 S_1 A_1 | B_1 S_1 B_1 | \varepsilon$

$A_1 \to a$

$B_1 \to b$

$\}$

Now $L = L_1^*$ can be represented by a grammar G such that

$G_1 = \{V, \Sigma, P, S\}$

$V = \{S, S_1, A_1, B_1\}$

$P = S \to S_1 S \mid \varepsilon$

$S_1 \to A_1 S_1 A_1 \mid B_1 S_1 B_1$

$A_1 \to a$

$B_1 \to b$

}

Thus grammar G is a context-free grammar and language L produced by G is also context-free language. Hence context free language are closed under kleen closure.

Theorem 4

If L_1 and L_2 are two CFLs then $L = L_1 \cap L_2$ may be CFL or maynot be CFL. That means L is not closed under intersection.

Proof Let $L_1 = \{a^n b^n c^i \mid n \geq 1, i \geq 1\}$

$L_2 = \{a^i b^n c^n \mid n \geq 1, i \geq 1\}$

The grammar for L_1 is

$S \to AB$

$A \to aA_1 \mid aL$

$B \to cB \mid c$

Similarly L_2 can be represented by grammar

$S \to AB$

$A \to aAa$

$B \to bb$

$L_1 = \{a^n b^n c^m \mid n, m \geq 1\}$

$L_2 = \{a^m b^n c^n \mid n, m \geq 1\}$

$G_1(L_2)$

$S \to AB$

$A \to aAb \mid ab$

$B \to cB \mid c$

$G(L_2)$

$S \to AB$

$A \to aA \mid a$

$B \to bBc \mid bc$

$L_1 \cap L_2 = a^n b^n c^n$ which is not CFL.

Now if we try to obtain

$L = L_1 \cap L_2$ then we get sometime context-free languages and sometime non-context free languages. Thus we can say that CFLs are not closed under intersection.

Theorem 5 If L_1 is a CFL then L_1' may or maynot be CFL. That means CFL is not closed under complement.

Proof Let L_1 and L_2 are two CFLs. We will assume that complement of a context free language is a CFL itself. Hence L_1' and L_2' are both CFLs. We can also state that $(L_1' \cup L_2')$ is context free. But $L_1' \cup L_2' = L_1 \cap L_2$ i.e $L = L_1 \cap L_2$ may or maynot be CFL. The L_1 and L_2 are arbitrary CFLs, there may exists L_1' and L_2' which are not CFL. Hence, complement of certain language may be context-free or may not be. Therefore, we can say that CFL is not closed under complement operation.

* Decision algorithm for CFLs

⇒ There are many questions about CFL that are undecidable that mean there doesnot exist any algorithm to answer these questions. Such questions are called the questions about undecidable context free Languages

1. Whether or not two different context-free languages define the same language?
2. Whether given CFL is ambiguous or not?
3. Whether complement of given CFL is context free language?
4. Whether the intersection of two context free language is a context free?

⇒ There is no algorithm to answer these questions. But there is a certain set of questions for which the answer can be obtained. Such questions are about context free language that are decidable

⇒ following is a list of those questions

1. Does the CFG G generates any string ?(emptiness)
2. Is the language generated by G is finite or not? (finiteness).
3. Suppose G is a grammar and W is a string, does G generates W? (membership).